Backpropagation Through Time (BPTT) and Gradient Issues

1 Backpropagation Through Time (BPTT)

1.1 Task 1: Gradient Computation

In a Vanilla Recurrent Neural Network (RNN), the parameters are shared across all time steps. Hence, gradients must be accumulated over the entire sequence.

Let:

$$V = W_{hy}, \quad W = W_{hh}, \quad U = W_{xh}$$

The gradients are computed as follows:

Output Layer

$$\frac{\partial L}{\partial V} = h_T \cdot \delta_y^T$$

Recurrent Weights

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \delta_t \cdot h_{t-1}^T$$

Input Weights

$$\frac{\partial L}{\partial U} = \sum_{t=1}^{T} \delta_t \cdot x_t^T$$

where:

$$\delta_t = \frac{\partial L}{\partial h_t}$$

1.2 Task 2: Chain Rule Through Time

Gradients are propagated backward through time using the recursive relation:

$$\delta_t = \left(W_{hh}^T \delta_{t+1}\right) \odot \left(1 - h_t^2\right)$$

This arises from repeated application of the chain rule:

$$\frac{\partial L}{\partial W} = \sum_{k=1}^{T} \frac{\partial L}{\partial h_T} \left(\prod_{j=k+1}^{T} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W}$$

In practice, this is efficiently implemented using a reverse loop:

```
for t in reversed(range(T)):
```

At each time step, gradients are accumulated while propagating the error backward.

1.3 Task 3: Exploding Gradients and Clipping

Due to repeated multiplication of Jacobians:

$$\prod_{j=k+1}^{T} \frac{\partial h_j}{\partial h_{j-1}}$$

gradients can grow exponentially, leading to exploding gradients.

To stabilize training, gradient clipping is applied:

```
np.clip(grad, -clip, clip, out=grad)
```

This ensures gradients remain within a bounded range and prevents numerical instability such as NaN losses.

1.4 Implementation: Backward Pass

```
def backward(self, cache, y_true, clip=5.0):
    X = cache["X"]
    h = cache["h"]
    probs = cache["probs"]

    T = X.shape[0]

    # Initialize gradients
    dW_xh = np.zeros_like(self.W_xh)
    dW_hh = np.zeros_like(self.W_hh)
    db_h  = np.zeros_like(self.b_h)
    dW_hy = np.zeros_like(self.W_hy)
    db_y  = np.zeros_like(self.b_y)

    # Output gradient
    dy = probs.copy()
    dy[y_true] -= 1

    dW_hy += np.outer(h[T], dy)
    db_y  += dy

    # Gradient into hidden state
    dh = self.W_hy @ dy

    # Backpropagation through time
    for t in range(T-1, -1, -1):

        dt = dh * (1 - h[t+1]**2)

        dW_xh += np.outer(X[t], dt)
        dW_hh += np.outer(h[t], dt)
```

```
        db_h  += dt

        dh = self.W_hh @ dt

    # Gradient clipping
    for grad in [dW_xh, dW_hh, db_h, dW_hy, db_y]:
        np.clip(grad, -clip, clip, out=grad)

    return {
        "W_xh": dW_xh,
        "W_hh": dW_hh,
        "b_h": db_h,
        "W_hy": dW_hy,
        "b_y": db_y
    }
```

2 CNN Backpropagation

2.1 Given

The binary cross-entropy loss is:

$$L = -\left[y \log \hat{y} + (1-y) \log(1-\hat{y})\right]$$

where:

$$\hat{y} = \sigma(Z_3)$$

The gradient at the output layer is:

$$\frac{\partial L}{\partial Z_3} = \hat{y} - y = \delta_3$$

2.2 Part A: Fully Connected Layer (W, b)

Forward Pass

$$Z_3 = W^T F + b$$

where $W \in \mathbb{R}^{8\times 1}$ and $F \in \mathbb{R}^{8\times 1}$.

Gradients

$$\frac{\partial L}{\partial W} = F \cdot \delta_3$$

$$\frac{\partial L}{\partial b} = \delta_3$$

Backpropagation

$$\frac{\partial L}{\partial F} = W \cdot \delta_3$$

Note: The weight gradient corresponds to an outer product (vector scaling).

2.3 Part B: Convolution Layer 2 $\left(W_k^{(2)}, b_k^{(2)}\right),\ k = 1, 2$

Step 1: Reshaping

$$\frac{\partial L}{\partial A_2} = \text{reshape}\left(\frac{\partial L}{\partial F}\right) \in \mathbb{R}^{2\times 2\times 2}$$

Step 2: ReLU Backward

$$\frac{\partial L}{\partial Z_2} = \frac{\partial L}{\partial A_2} \odot \mathbf{1}(Z_2 > 0) = \delta_2$$

Step 3: Gradients

Weights

$$\frac{\partial L}{\partial W_k^{(2)}} = P_1 \star \delta_2^{(k)}$$

Bias

$$\frac{\partial L}{\partial b_k^{(2)}} = \sum_{i,j} \delta_2(i, j, k)$$

Step 4: Backpropagation to P_1

$$\frac{\partial L}{\partial P_1} = \sum_{k=1}^{2} \delta_2^{(k)} * \text{rot180}\left(W_k^{(2)}\right)$$

Note: This is a full convolution with rotated filters.

2.4 Part C: Convolution Layer 1 $\left(W^{(1)}, b^{(1)}\right)$

Step 1: MaxPooling Backward

$$\frac{\partial L}{\partial A_1} = \text{Upsample}\left(\frac{\partial L}{\partial P_1}, M\right)$$

Note: Only the maximum locations receive gradients.

Step 2: ReLU Backward

$$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \odot \mathbf{1}(Z_1 > 0) = \delta_1$$

Step 3: Gradients

Weights

$$\frac{\partial L}{\partial W^{(1)}} = X_{\text{pad}} \star \delta_1$$

Bias

$$\frac{\partial L}{\partial b^{(1)}} = \sum_{i,j} \delta_1(i, j)$$

Note: Padding must be considered during gradient computation.

3 Time Complexity: Transformer vs RNN

3.1 1. Transformer Time Complexity

For a Transformer encoder layer, the core self-attention operation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Step-by-step Cost

- Computing QK^T: $\mathcal{O}(L^2 \cdot d)$
- Softmax: $\mathcal{O}(L^2)$
- Multiplication with V: $\mathcal{O}(L^2 \cdot d)$

Dominant Complexity:

$$\mathcal{O}(L^2 \cdot d)$$

3.2 2. RNN Time Complexity

At each timestep, the hidden state update is:

$$h_t = Wx_t + Uh_{t-1}$$

Cost Analysis

- Cost per timestep: $\mathcal{O}(d^2)$
- For sequence length L: $\mathcal{O}(L \cdot d^2)$

3.3 3. Ratio Comparison

We compare the complexities:

$$\frac{\text{RNN}}{\text{Transformer}} = \frac{L \cdot d^2}{L^2 \cdot d}$$

Simplifying:

$$= \frac{d}{L}$$

Numerical Substitution

$$L = 1024, \quad d = 256$$

$$\frac{d}{L} = \frac{256}{1024} = \frac{1}{4}$$

Final Result:

$$\frac{\text{RNN}}{\text{Transformer}} = \frac{1}{4}$$

Interpretation: For these values, the RNN is asymptotically 4× cheaper than the Transformer in terms of time complexity.

4 RNN Recurrence and Gradient Analysis

4.1 Given Recurrence

$$h_t = Ah_{t-1} + Bx_t$$

4.2 1. Expanding h_t in Terms of h_0

Unrolling the recurrence:

$$h_t = A^t h_0 + \sum_{i=1}^{t} A^{t-i} B x_i$$

Final Expression:

$$h_t = A^t h_0 + \sum_{i=1}^{t} A^{t-i} B x_i$$

4.3 2. Gradient Propagation

We analyze how gradients propagate through time:

$$\frac{\partial h_t}{\partial h_{t-k}}$$

From the recurrence, each step contributes a multiplication by A. Therefore, over k steps:

$$\frac{\partial h_t}{\partial h_{t-k}} = A^k$$

4.4 3. Spectral Radius Analysis

Let:

$$\rho(A) = \max |\lambda_i|$$

where λ_i are the eigenvalues of A.

Case 1: $\rho(A) > 1$

$$A^k \to \infty \quad \text{as } k \to \infty$$

Gradients grow exponentially, leading to:

Exploding Gradients

Case 2: $\rho(A) < 1$

$$A^k \to 0 \quad \text{as } k \to \infty$$

Gradients shrink to zero, leading to:

Vanishing Gradients

5 State Space Model (SSM) Analysis

5.1 Given System

$$h_k = Ah_{k-1} + Bx_k, \quad y_k = Ch_k$$

5.2 1. Deriving y_k in Terms of Inputs

Step 1: Expanding Hidden States

$$h_1 = Ah_0 + Bx_1$$

$$h_2 = Ah_1 + Bx_2 = A^2h_0 + ABx_1 + Bx_2$$

$$h_3 = A^3h_0 + A^2Bx_1 + ABx_2 + Bx_3$$

Step 2: General Form

$$h_k = A^kh_0 + \sum_{i=1}^{k} A^{k-i}Bx_i$$

Step 3: Compute Output

$$y_k = Ch_k$$

Substituting:

$$y_k = CA^kh_0 + \sum_{i=1}^{k} CA^{k-i}Bx_i$$

Step 4: Ignoring Initial State

Assuming $h_0 = 0$:

$$y_k = \sum_{i=0}^{k} CA^{k-i}Bx_i$$

5.3 2. Convolution Interpretation

Compare with 1D discrete convolution:

$$y_k = \sum_{i=0}^{k} K_{k-i}x_i$$

where the kernel is:

$$K_{k-i} = CA^{k-i}B$$

Final Interpretation:

$$y_k = (K * x)_k$$

Thus, the system behaves like a 1D convolution over the input sequence.

5.4 3. Intuition

- Each input x_i influences future outputs.
- The influence decays through powers of A^{k-i}.
- The kernel K captures the memory of the system.

5.5 4. Time-Invariance

A system is time-invariant if a shift in input results in an equivalent shift in output.

Justification

- A, B, and C are constant matrices.
- The kernel:
$$K_{k-i} = CA^{k-i}B$$
depends only on the relative index $(k-i)$.

Final Conclusion:

The system is time-invariant since the response depends only on $(k-i)$.

6 Selective State Space Model (Selective SSM)

6.1 Given Model

$$h_k = Ah_{k-1} + Bx_k$$
$$y_k = g_k \odot (Ch_k), \quad g_k = \sigma(W_g x_k)$$

6.2 1. Dimension Verification

Let:
$$x_k \in \mathbb{R}^{d_{\text{in}}}, \quad h_k \in \mathbb{R}^d, \quad y_k \in \mathbb{R}^{d_{\text{out}}}$$

Hidden State Update

$$Ah_{k-1} \Rightarrow (d \times d)(d) = d$$
$$Bx_k \Rightarrow (d \times d_{\text{in}})(d_{\text{in}}) = d$$

$$\Rightarrow h_k \in \mathbb{R}^d$$

Output Computation

$$Ch_k \Rightarrow (d_{\text{out}} \times d)(d) = d_{\text{out}}$$

Gate Computation

$$g_k = \sigma(W_g x_k)$$
$$W_g x_k \Rightarrow (d_{\text{out}} \times d_{\text{in}})(d_{\text{in}}) = d_{\text{out}}$$

$$\Rightarrow g_k \in \mathbb{R}^{d_{\text{out}}}$$

Final Output

$$y_k = g_k \odot (Ch_k)$$

Both terms are in $\mathbb{R}^{d_{\text{out}}}$, hence element-wise multiplication is valid.
Conclusion: All dimensions are consistent.

6.3 2. Time-Invariance Analysis

For a time-invariant system:

$$y_k = \sum K_{k-i} x_i$$

However, here:

$$y_k = g_k \odot (Ch_k), \quad g_k = \sigma(W_g x_k)$$

Key Observation:

- The gate g_k depends explicitly on the current input x_k.
- Thus, the output depends on input *content*, not just time difference.

Conclusion:

The system is NOT time-invariant since g_k depends on x_k.

6.4 3. Output Derivation

From the standard SSM expansion:

$$h_k = \sum_{i=0}^{k} A^{k-i} B x_i$$

Applying the output equation:

$$y_k = g_k \odot (Ch_k)$$

Substituting h_k:

$$y_k = g_k \odot \left(\sum_{i=0}^{k} C A^{k-i} B x_i \right)$$

Final Expression:

$$y_k = g_k \odot \left(\sum_{i=0}^{k} C A^{k-i} B x_i \right)$$

6.5 4. Why It Is Not a Fixed Convolution

For a standard SSM:

$$y_k = \sum K_{k-i} x_i, \quad K_{k-i} = CA^{k-i}B$$

This represents a fixed convolution kernel.

However, in Selective SSM:

$$y_k = g_k \odot \left(\sum K_{k-i} x_i \right)$$

Since:

$$g_k = \sigma(W_g x_k)$$

the effective kernel becomes:

$$K_{k-i}^{(\text{dynamic})} = g_k \cdot (CA^{k-i}B)$$

Key Insight:

- The kernel varies with x_k at each timestep.
- Hence, it is input-dependent and changes over time.

Final Conclusion:

The system cannot be represented as a fixed convolution since the kernel is dynamic.

7 Model Comparison and Analysis

7.1 1. Performance Degradation with Sequence Length

The LSTM model degrades the most as sequence length increases.

This is primarily due to the vanishing gradient problem, which makes it difficult for LSTMs to retain information over long contexts. As the sequence length grows, earlier inputs have diminishing influence on the hidden state, leading to poorer performance.

Conclusion: LSTMs exhibit the most degradation for long sequences due to limitations in long-range dependency modeling.

7.2 2. Selective SSM vs LSTM and Transformer

Selective SSM generally outperforms standard Linear SSM and can outperform LSTMs on long sequences due to its input-dependent gating mechanism, which allows adaptive control over information flow.

However, in the current experiments, the results are inconsistent due to limited training, initialization sensitivity, or insufficient convergence. Therefore, no definitive numerical conclusion can be drawn.

Conclusion:

- Empirically: Results are inconclusive due to instability.
- Theoretically: Selective SSM offers a strong trade-off between efficiency and performance.

7.3 3. Failure Case of Transformers on Long Sequences

A key limitation of Transformers is their quadratic time and memory complexity with respect to sequence length.

$$\mathcal{O}(L^2 \cdot d)$$

This results in:

- High memory consumption
- Slower training and inference for long sequences (e.g., $L = 1024$)

Conclusion: Transformers become inefficient for very long sequences compared to linear-time models such as SSMs.